

American Sign Language Captioning - Report

(Submission for Shreyas Bhat)

Shreyas Bhat

Jyotirmoy Mukherjee

Abstract

Sign Language understanding poses several interesting computer vision problems. These are broadly classified into Sign Language Detection, Sign Language Recognitions and Sign Language Translation. Here we propose American Sign Language to English, a comprehensive fast sign language recognition system for ASL. We approach this as a gesture recognition problem inspired by the Human Activity Recognition using Pose machines. The present study comprises of experiments carried out on the Human Activity Recognition Dataset for classifying 6 activities. The framework developed is then used as a two-part solution to the task of learning American Sign Language. In the first step we use a pose machine to extract hand and face pose features. The obtained features are then used to train Recurrent Neural Networks to learn Sign to Word glossary. The network can then use Realtime pose information to predict ASL words and produce captions.

Introduction

Understanding Sign Language is a challenging task since it requires the ability to learn both visual information and language to recognize the meaning. Hence it is important to understand the distinction between Sign Language Translation and Sign Language Recognition tasks. SLT refers to not only recognizing but also language translation, such as, German Sign Language to English and SLR refers to recognizing the words in the same language, such as ASL to English [1][2]. However, SLR can still be treated as a translation task as ASL and English are grammatically and structurally different from each other. This is especially important since we use the word translation in place of recognition going forward [3].

While the goal of the project is to produce a real-time ASL to English translation system which can be broken into three steps; 1) Track pose and obtain frame-level pose vectors, 2) Learn Word level classification of series of poses 3) Learn to form grammatically correct written English sentences [2]. We limit the scope of this project to

the first 2 steps for the sake of developing a comprehensive and easily reproducible to package.

Before we start, we need a exhaustive labelled dataset to build a vocabulary of Signs, as a part of this project we have compiled a comprehensive dataset from Word Level American Sign Language (WLASL)[4] and Microsoft American Sign Language (MSASL)[5] datasets and produced the tools to extract and process the files as required for this task.

Since we are approaching this as a gesture recognition problem, we first develop a model on a gesture recognition dataset [6] called the Human Activity Recognition (HAR Dataset) to understand the framework of classifying activities using pose information. Next, we generate the pose data-frame for sign language. Advances in real-time pose networks has enabled their use tasks such as gesture and action recognition. While using pose tracking to learn word-level sign language isn't a new approach [7], our network is different in that we use both hand and face tracking unlike the previous works. Since sign language makes use of both hand and facial information, it is hypothesized in this project that pose information generated from face and hand landmarks can be used to enhance sign language translation. The second step is using a vectorized form of these tracked features to train a Recurrent Neural Network.

We have also produced a package which will be made available on request at our GitHub¹ page (since the package still in development, once complete it will be made available publicly) along with a Colab notebook² to help use the code. Additionally. All the network development and testing work was conducted on a separate notebook³ which is added in the footnote here. In the next section we will discuss the related works each part of the approach in detail.

Related Works

The early works on sign language translation using computer vision techniques struggled because they were limited to training on images which resulted in use of only static signs

¹ https://github.com/bshreyas13/ASL_ACV_Project

² <https://colab.research.google.com/drive/12tf0IT1REaxJxdUNvtE64lrbxogdNehx?authuser=1#scrollTo=IAuADkxR5djU>

³ <https://colab.research.google.com/drive/10VZgj0n4-fH-VTUdmA6YHFdup0yLLLut?authuser=1#scrollTo=GPO9eHJOdW1c>

to learn an inherently dynamic language[9]. However, Sign Language Translation using video data introduces the ability to translate dynamic signs. Successful results have been achieved in SLT and SLR even going as far as SLT and SLR combined to recognize and translate German Sign Language to English [9]. This trend is leading toward the use of spatial and temporal information for classification of signs. The state-of-the-art in this domain is built on a sentence level annotated German sign language dataset which limits its direct application to ASL to English. Further comprehensive world-level datasets have been proposed for ASL[4][5] in the recent years which opens doors for using concepts like action recognition from pose [10]. Our method used an RNN to classify hand gestures as signs [11] and predict the word level translation.

Dataset:

The dataset and tools used here is built on the annotated data and tools provided by WL-ASL and MS-ASL which are two word-level ASL datasets. The combined set contains 20,709 word-level ASL video samples spanning over 2500 unique words. For the experiments outline here we have used a subsampled dataset compiled with examples that have more than 10 videos for the corresponding labels.

Experimental Methods:

1. Human Activity Recognition

For sanity check and proof of concept, we trained our networks on the Human Activity Recognition (HAR) dataset which is available online [18]. The data consists of 6 labels and 1438 videos that were already converted into 2D pose vectors using OpenPose as shown in Figure 1. This experiment gave us a baseline on the effectiveness of the LSTM time series analysis approach we are taking to build the Glossary network. Further, we also gained insights on proper data preprocessing required to learn time series sequence of poses. Here we used a stacked LSTM model of 3 layers with 512, 256 and 128 hidden units respectively, followed by a dense layer of 100 units and a Dense-SoftMax layer for prediction. This model was trained on the HAR dataset and the results are discussed in the following sections.

HAR Real Time Implementation:

To further illustrate the efficiency and effectiveness of Pose + LSTM approach we implemented a real time pipeline that takes a webcam stream as input and used our trained model to predict the activity as one of the 6 classes (eg: waving, boxing, clapping). The code for the same is available on our

GitHub⁴. The implementation does not require a GPU and can be run on relatively low powered CPU.

2. American Sign Language

The challenges with learning sign language are having a high-quality annotated dataset and an efficient model. In the following sections, we discuss the experiments that were carried out to understand and analyze the different deep networks that can perform the task at hand. There are two broad parts of experiments in this study: 1. Dataset preparation and preprocessing that involves data augmentation methods, increasing number of frames through overlap, scaling, and weighing pose information etc. 2. Recurrent Neural Networks: we experimented for two RNN based Deep networks namely, a stacked LSTM model and a Y-LSTM Network. These networks were optimized by analyzing the effects of activation functions, optimizers, number of layers and hidden units and various regularization.

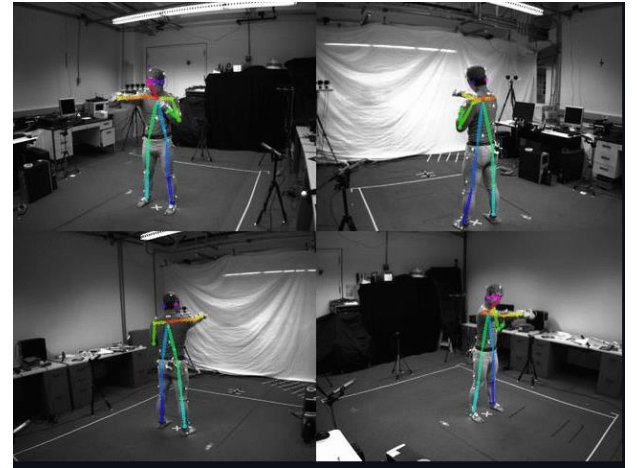


Figure 1: HAR dataset - Pose tracking for activity recognition^[17]

Dataset Generation: Pose Tracking with Mediapipe

The hand and face pose vectors were obtained using pretrained networks from mediapipe[14] an open-source ML framework made available by Google Inc. It is implemented on tensorflow-lite since it is a realtime pose machine for mobile platforms. It is also ideal for realtime ASLE due to fast inference time and light memory consumption. We combined their hand pose tracker and face pose tracker to build a combined hand and face tracker. Some examples are illustrated in Fig 4.

The mediapipe pose machine used single shot multibox detector for object detection[15] and uses the face

⁴ https://github.com/bshreyas13/Realtime_HAR

landmarks approach for face detection[16]. Though the network is designed individually for hand pose and face pose we were able to write a combined face and hand tracker using the mediapipe python package.

We set all videos to $n_step=60$ frames as the average length of signs were around 2 seconds. Each frame was resized to 256×256 . For each frame of the video, the pose tracker produces 21 hand-tracking points and 468 face-tracking points. It gives 4D points (x,y,z,v) . The z coordinate is obtained by projecting the 2D image orthogonally into 3D a geometrical space. Each keypoint is represented by a factor v , which is the *visibility* parameter that accounts for occlusions or overlap as we can see in Fig4c. For more robust tracking and to deal with handedness-bias the sample are flipped and tracked. This data augmentation step is built into mediapipe and works in our favor as the network must be invariant to the handedness of the signers. Based on literature and several experiments, it was

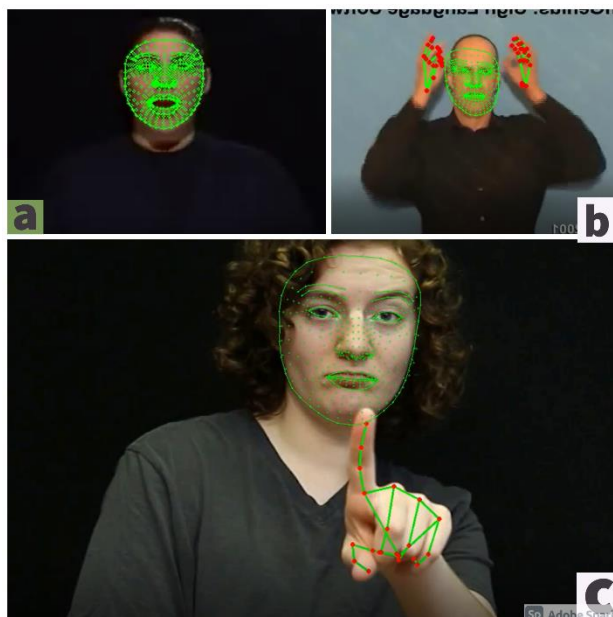


Figure 2: Hand+Face Pose Tracking with Medipipe.(a) Sample of video where hand enters the frame late, (b) Tracking of low quality footage, (c) Sample case for good tracking

found that only the (x,y,z) coordinates of the pose play a significant role in the classification process. Hence, the information used for this study is limited to 3D coordinate information.

Besides the cases where the pose machine completely fails, there were still edge cases where the hand enters after a few frames (Fig4a) and cases where hand tracking was not consistent due to poor quality sample (Fig4b). We addressed these issues by padding the features with zeros for those frames which then gives us frame-level set of key-

points associated with each video and mapped with its label. The Labels are then encoded using OneHotEncoder before vectorizing.

Frame-Level Pose Vectors and Data frames

We then turn the extracted frame-level keypoints into a vector descriptor of poses for $n_steps=60$ frame. From this we obtain a vector descriptor of series of poses for each video of $dim(1, n_steps, (21+468)*4)$ which are concatenated to form videos vice vector descriptors.

Since no off the shelf functions were available from open source libraries, user defined programs were written to generate the feature vectors as a dataframe. We used our GenerateTensor Toolkit to Process the video data and extract $X_{train/test}$ and $Y_{train/test}$ sets of pose vectors. We also ran it on for the 18K samples in WLASL dataset. Both of which can be made available for download on request.

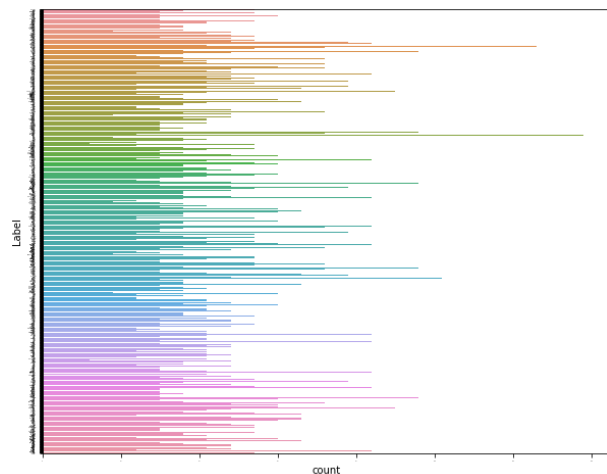


Figure 3: Sample ASL data distribution

Data Preprocessing:

Once the pose vectors are obtained, the dataset is tuned for several experiments to study its effect on the model's classification accuracy. Before we started, we dropped the visibility parameter which we used in our previous works on the project.

1. **3D co-ordinates (Hand+Face):** We experimented first with 3D pose. In this method, all (x,y,z) information were used for the landmarks and the models were trained. The vectors used here were a concatenated version of the pose information obtained for both face and hand.
2. **2D Co-ordinates (Hand+Face):** The next variation on the dataset was using 2D pose. Since

a human eye can differentiate pose on an image using just 2D coordinates, the idea that computers could also have a similar capability was tested. For this purpose, the z coordinates were dropped from every tuple. The dataset here again contained the concatenated pose of both hand and face.

3. **Hand Pose only:** Another observation that was made from visual inspection of the videos were that, while facial information does play an important role, hand information was more significant in the temporal domain, that is, hand movements were more obviously discernable in comparison to facial movements. Since the pose level information is used here, it was realized that the change in coordinates in the facial pose were far lesser than relative change in the hand pose. The idea was to weight the pose level of information for face and hand separately, however, in this part of the experiment, only hand level features were used to study its effect on the learning.
4. **Parameterized Hand and Face Pose:** From the above experiment it was seen that there was a positive effect on learning when using only hand pose. This inspired the next experiment of training a Y-LSTM-network (discussed in RNN section) which takes the hand and face pose as separate streams. For this purpose, the dataset was modified to separate the hand and face level information.
5. **Overlap/No Overlap:** The dataframe as mentioned earlier consists of 60 frames per video. In one experiment, the network was trained on a sequence of 60 frames, however, sequential models perform better when several samples of sequences are provided for a given class. For this purpose, the 60 frames were windowed for several sizes like 20, 30,40 of which a window of size 30 seemed to have better performance. The sequence was built such that, there was an overlap of p% across the windows. So, for example, if there is an overlap of 50%, then the sequence contains the first 30 frames to which 50% of the previous window and 50% of the next window is appended and so on thus resulting in 90 frames for each sample. Several research approaches [18] emphasized on the overlapping of sequence and the experiment corroborated the previous claims.
6. **Binary Classification Dataset:** Since the dataset for ASL used here had lower samples per label, the idea was to investigate the algorithm performance

on binary classification as an exaggerated case. This was carried out to study the capability of different RNN models like stacked LSTM, ConvLSTM, Y-Network models etc.

Recurrent Neural Networks:

Sequential learning models take an input of dimension (*samples, n_steps, features*) where samples here refer to each video, *n_steps* refer to number of frames and *features* refer to the pose vector. Two models were studied: 1. Stacked LSTM Model 2. Y-Network model.

1. Stacked LSTM Model:

Once we obtained the vectors, we move to step two of our approach. We used these descriptors to a stacked LSTM multiclass classifier which we call the Glossary Network. This part of the overall network is intended to learn Word-to-Sign associations which can then be used to produce sentence level outputs. Fig4 shows the network architecture used for the Stacked LSTM. The network takes the input *dim (n_samples, n_steps, (21+468)*3)*. The network produces a video-level class prediction which we use to train it with RMSprop optimizer, Categorical cross entropy loss. The LSTM network was tested on several optimizers like adam, SGD, adagrad etc and it was seen that the RMSprop optimizer works most efficiently. Different activation functions were also tested like relu, sigmoid, tanh etc and the effect of tanh surpassed the others.

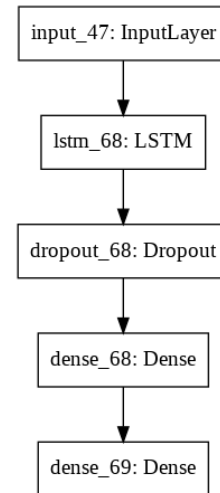


Figure 4: Stacked LSTM Network Model

2. Y-LSTM Network:

As mentioned earlier, the idea that hand gestures are more discernible than face gestures inspired the experiment of training the network separately for hand and face pose. To this end, a Y Network was built as shown in the figure5. The left network took the hand pose as input and was made with stacked LSTMs while the right model took the face features as input and was built using a single LSTM layer and dense layer. Like the previous section the choice of activation functions for the hand and face gesture models played a significant effect on the classification.

Experimental Results and discussions:

1. Human Activity Recognition

The Stacked LSTM model for the HAR dataset gave a training accuracy of 100% and a test accuracy of 99%. The loss curves and accuracy in figure6 curves agree with the results in the source paper [16]. From this result, it was understood that the framework of using pose for activity recognition is a valid approach and as it has been verified on the previously tested.

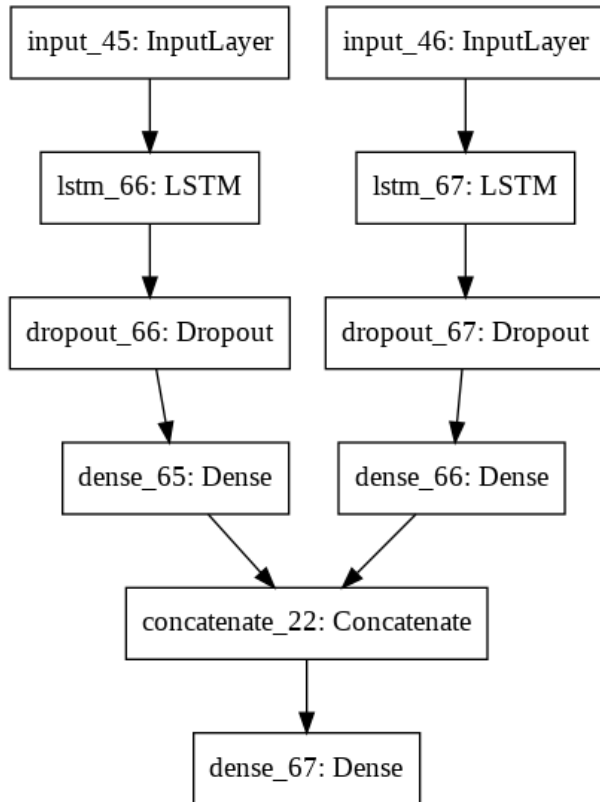


Figure 5: Y-Network model

HAR dataset. Further, since the model was newly built and gave such high accuracies, it was learnt that the dataset for ASL must be investigated. The idea was to develop an ASL dataset that was like HAR. The model pipeline was also implemented in real time. The images were fed from the

webcam and the action was recognized as seen in the figure7.

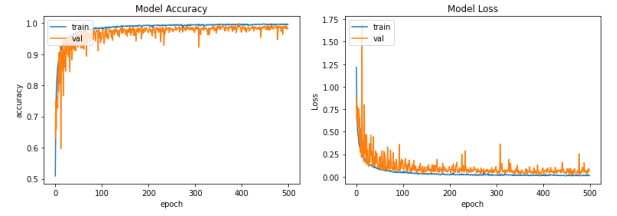


Figure 6: Model and Accuracy plots for HAR dataset

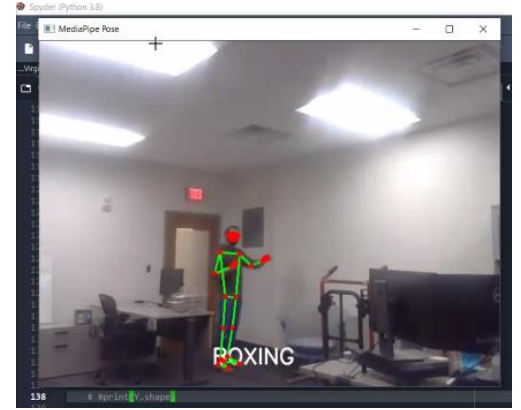
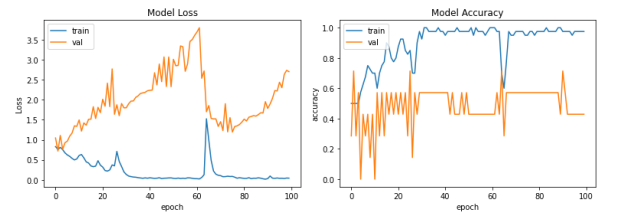


Figure 7: Real time implementation of HAR Model using webcam for input stream.

1. American Sign Language

We started with 3D pose to train the LSTM stacked model for 10 classes with 10 videos for each class. Here we saw the training accuracy reach 100% however, the validation accuracy was near 33%. The plots can be seen in Experimental Curves1 below.

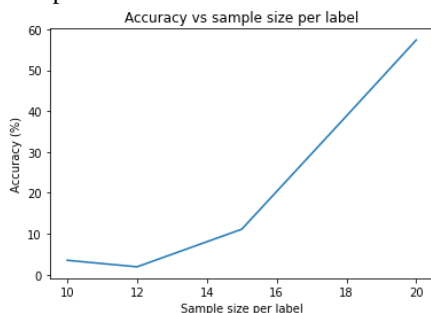


Experiment Curves 1: Accuracy and Loss curves for 3D pose stacked LSTMS.

Initially, we assumed the model is not tuning and hence is overfitting and after several iterations of using overfitting reduction techniques, we realized that tuning the model made difference. We then went back to the HAR dataset which was built from 1438 videos for 6 labels which results on an average of 240 videos per label. However, since the present project considered building the dataset from videos suggested by WSASL, the number of videos per label are

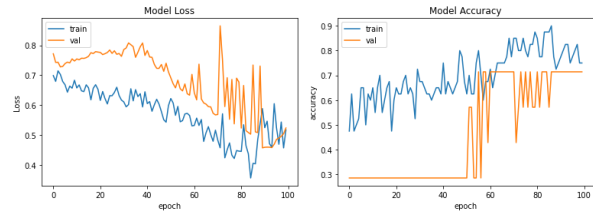
limited by what is presented in the original paper, which is much lower than the ratio of the HAR set. The ASL dataset created from the

pose machine was built with all samples with more that 10 videos per label on an average which is nearly 96% lower than HAR. Although, the WLASL set has ample total samples, yet the training videos per sample is quite low. This was investigated by trying to reduce the number of samples and use maximum available videos per sample. During further exploration it was seen that the dataset contains 20 videos for certain labels and 3 videos only for some! This skew in the distribution was a major reason for the low accuracies obtained in the validation. This is corroborated by the results on the HAR dataset shown preciously. With this insight, we tested the model on a metric no. of videos / no. of labels vs accuracy which can be seen in figure. The following figure helps to understand how the accuracy depends on the dataset i.e., number of available samples to the number of labels.

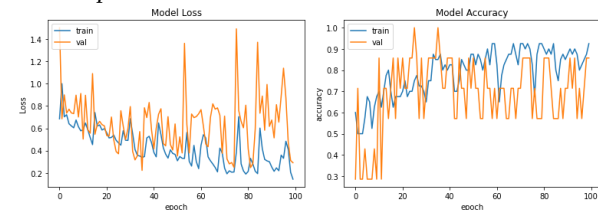


Experiment Curves 2: Videos/label vs accuracy

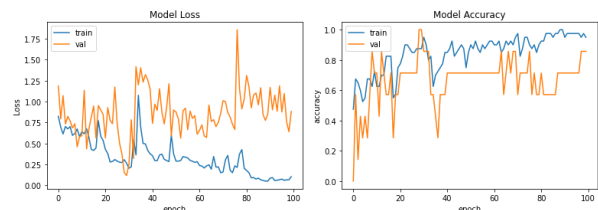
Since it was now established that the number of videos is significantly small, it was better to reduce the problem to a binary classification problem and study the effect of other parameters such as the overlap size, model tuning etc. The figures below show the model loss and accuracy curves for the experiments that were carried out. It was seen that 2D pose (figure3 and 4) information in general performed better than 3D information (figure 1). Further, when the hand gestures alone were fed into the network, the model was able to track the accuracy and loss much better than the others. Another model, the Y-network was then trained to deal with the face and hand pose information separately. The model learning trend seemed better than the others as it was very sensitive to the face features and hand features. The Y-network gives the flexibility of tuning the hand and face learning differently. The test accuracy in Y-network was 87% as seen in Experiment Curves 5.



Experiment Curves 3: 2D Hand + Face Curves



Experiment Curves 4: 2D Hand only



Experiment Curves 5: Y-LSTM network curves

Computing Resources

The present network and prototype code is built for compatibility with the latest tensorflow-gpu and tflite packages which severely limited our computing resources as we were still unable to get ARC to work with tf2. All the above was tested across Google Colab and one Intel 12 core multi-threaded windows CPU severely limiting our computing capacity to deal with the large Dataset. It is worth noting however, that the tests on Colab are best run on a small randomly sampled dataset. The primary challenge with processing large datasets is availability of RAM for which we optimized our code by having using data compression at all intermediate outputs and breaking down the processing into chunks as we extracted and constructed the pose feature vectors. This makes the training process slower however, it also allowed us to optimize the prototype as best as possible. We would like to note that updated HPC resources would greatly assists with these tasks.

Discussion

The goal of this project was to accurately translate the American Sign Language. The framework was motivated by the Human Activity Recognition carried out by Stuart et al.[17] which we also tested on our Stacked LSTM Network

as part of this project. The next task was to use this model to learn the ASL dataset. From the experiments above, there were significant learnings that helped improve the accuracy from near 1% to 45%. Firstly, it was understood that overlapping sequences for LSTM networks improves the learning process, secondly having a well-balanced dataset improves learning process. In most applications, it is arduous to have a uniformly distributed data however, subsampling could be useful in such cases or sifting through high frequency information. Another learning was that the number of samples per label must be high for any supervised learning approach. To place an exact number on what number is the best would be hard, however, at least 100 videos for each label in this project could improve results. It was seen that 2D pose outperformed 3D pose which we presume is because of the quality of the depth estimates. Also, both face and hand gestures are important in sign language however, the relative displacement in the hand coordinates are far more significant than the face gestures. A YLSTM-network was made based on this understanding; this allows for fine grained control of the influence of each of pose vectors on the overall learning and tune the network for better performance.

Conclusions and Future Steps

The development of the LSTM model for HAR dataset gave an initial insight into developing the dataset for ASL. It was important to note that having the right data preparation is an important part of creating working deep learning models. We also established using HAR that the approach can be effectively used for realtime application. However, further analysis of training and inference time would yield a clear metric for the observation made from the experiments. We are limited by time to acquire and plot these metrics. This study accomplished the following:

1. Development of model for HAR dataset with 99% validation accuracy.
2. Real time system that used the model trained on HAR to predict activity by taking in webcam stream.
3. Developed a pose dataframe for American Sign Language.
4. Developed models to accurately learn the pose for video level information to predict the correct word label for a given action. 33% accuracy was achieved in multi-class classification (10 labels) and 87% for binary classification due to limited dataset.
5. The experiments also showed us the extremely low training times of the network when accurately tracked sparse features are used for learning.

From the experiments it was seen that when the number of labels were reduced and samples were increased, the learning improved, and accuracies reached 87%! This leads

to the future work of creating a new dataset for ASL for word level classification with larger number of videos per label.

Contribution

If I must point out contributions, I was the primary in setting up Realtime HAR, mediapipe Pose network, vectorizing pose features and setting up the Colab notebooks for use along with data preprocessing and scripting for the networks and Jyotirmoy did most of the testing tuning the Glossary Network and explored ideas like Y-LSTM, and Overlapping sequences etc. However, I would like to point out that the project was for the most part a collaborative effort Including several hours of peer programming both in person and over Zoom calls. Since we each created and debugged scripts and were involved in the ideation and experiments together since the first day it would be unfair to give undue credit to an individual.

References

1. Adaloglou et al. A Comprehensive Study on Sign Language Recognition Methods, CVPR 2020
2. Necati Cihan Camgoz et al. Sign Language Transformers: Joint End-to-end Sign Language Recognition and Translation, CVPR 2020
3. Saeideh Ghanbari Azar, Hadi Seyedarabi, Trajectory-Based Recognition of Dynamic Persian Sign Language Using Hidden Markov Model, arXiv 2019.
4. Dongxu Li, Cristian Rodriguez Opazo, Xin Yu, Hongdong Li, Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison, The Australian National University, Australian Centre for Robotic Vision (ACR) 21 Jan 2020
5. Hamid Vaiz Joze et al, MS_AS_L A Large Scale dataset and Benchmark for understanding ASL, BMVC 2019
6. Amit Moryossef et al. Real-Time Sign Language Detection using Human Pose Estimation, ICCV 2020
7. Dong et al. American Sign Language Alphabet Recognition Using Microsoft Kinect, CVPR 2015
8. Nicas, M., Best, D, A study quantifying the hand-to-face contact rate and its potential application to predicting respiratory tract infection, Journal of occupational and environmental hygiene 5(6), 347–352 (2008)
9. Necati Cihan Camgoz et al. Sign Language Transformers: Joint End-to-end Sign Language Recognition and Translation, CVPR 2020
10. Dennis Ludl, Thomas Gulde and Cristobal Curio, Simple yet efficient real-time pose-based action recognition, arXiv:1904.09140
11. Real-time Hand Gesture Detection and Classification Using Convolutional Neural Networks <https://arxiv.org/pdf/1901.10323v3.pdf>
12. Zhe Cao et al. OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields, IEEE 2019.
13. Fang et al. RMPE:Regional multiperson pose estimation. ICCV 2019

14. Google Inc, Mediapipe framework, <https://google.github.io/mediapipe/>, accessed Nov 2020
15. Wei Liu et al. SSD: Single Shot MultiBox Detector , CVPR 2016
16. Yury Kartynnik et al, Real-time Facial Surface Geometry from Monocular Video on Mobile GPUs, Google Research , arXiv 2019
17. <https://github.com/stuarteiffert/RNN-for-Human-Activity-Recognition-using-2D-Pose-Input>
18. https://atrium.lib.uoguelph.ca/xmlui/bitstream/handle/10214/9338/Sharma_Shilpy_201511_PhD.pdf?sequence=1&isAllowed=y